

COMP 333: Concepts of Programming Languages

Fall 2026

Instructor: Kyle Dewey (kyle.dewey@csun.edu)

Course Web Page: <https://kyledewey.github.io/comp333-fall26>

Office: JD 4419

Course Description (From the Catalog)

Discussion of issues in the design, implementation and use of high-level programming languages through a historical framework, including how languages reflect different design philosophies and use requirements and the technical issues in the design of main abstraction constructs of programming languages. Other approaches to imperative or object-oriented programming, functional programming, logical programming and parallel programming.

Learning Objectives:

Successful students will be able to:

- Understand when to use, and write programs using:
 - Execution approaches: compilation, interpretation, and just-in-time (JIT) compilation
 - Memory management: manual, reference counting, garbage collection, ownership and borrowing
 - Types: dynamic typing, static typing, strong typing, weak typing, untyped
- Write programs utilizing class-based and prototyped-based inheritance, virtual dispatch, and higher-order functions
- Write memory diagrams representing the connections between allocated values, differentiate between stack-allocated and heap-allocated values, and understand when to use stack vs. heap allocation.

Course Motivation and Goal

Programming languages, like the tools in a typical toolbox, are built to solve problems. A toolbox may have different sizes and shapes of both hammers and screwdrivers. Similarly, different programming languages may be closely related to each other, or potentially very different from each other. The more different a language or tool, the more different the kind(s) of problem(s) it is designed to solve.

The danger of getting too close to one language or programming paradigm is that your thinking adapts to fit that language/paradigm. In keeping with the toolbox analogy, if you have a hammer, then all problems become nails. I may insist on fixing a leaky pipe with a sledgehammer, but I won't get back my security deposit.

My primary goal with this course is to expand your toolbox, and expose you to languages which behave very differently from each other. My intention is to warp your brain a bit, and force you to think in ways you're not used to. This will improve your

problem-solving skills, specifically by allowing you to look at the same problem from different angles.

A secondary goal is to give you a sense of how different languages are built, and how they work. We will focus primarily on modern language design and implementation, though many basic concepts haven't changed much in the 60+ year history of programming languages.

Textbook and Other Required Class Materials

No textbooks are required. You may wish to look at Programming Language Pragmatics (Michael Scott) as a reference, though the course does not follow that book. A computer, be it a laptop or otherwise, is required.

Grading

Your grade is based on the following components:

Assignments	14%
Midterm Exam 1	27%
Midterm Exam 2	27%
Final Exam	32%

There will be a series of coding-based assignments issued throughout the semester, which cover core parts of the different languages and paradigms you'll use. Not all of these will be weighted evenly, nor will you always be given the same amount of time for assignments. Exactly which assignments are assigned depends on how the class progresses. In general, assignments will be submitted through Canvas (<https://canvas.csun.edu/>). In the event that there is a problem with Canvas, you may email your assignment to me (kyle.dewey@csun.edu) as a last resort.

Plus/minus grading is used, according to the scale below:

If your score is >=...	...you will receive...
92.5	A
89.5	A-
86.5	B+
82.5	B
79.5	B-
76.5	C+
72.5	C

If your score is >=...	...you will receive...
69.5	C-
66.5	D+
62.5	D
59.5	D-
0	F

If you are not present for the final exam and you have not previously made alternative arrangements with me for the final exam, a grade of WU (unauthorized withdrawal) will be assigned.

Collaboration for Assignments

All students are required to submit their own individual work. For assignments (and **only** assignments), students may discuss among each other, as long as they don't digitally share code. That is, you **cannot** simply email your code to someone else. However, you **may** discuss your actual code with someone else, including merely viewing code. The only stipulation is that **if you do discuss with someone else, say so in your submission**. This is not for punitive reasons; this is only so I get a sense of who is working with who. My intention with this policy is to enable collaborative learning, as opposed to simply sharing a solution.

Plagiarism and Academic Honesty

While collaboration is allowed on assignments, you are responsible for all of your own work. You may **not** take code from online sources and submit it as your own. If you must take code from online, cite where you took the code from. Worst-case scenario, you'll receive a 0 for whatever you took, but no further action will be taken. In general, code taken online which solves more general things (e.g., "how do I iterate through an array in Java") is more acceptable than code which solves more specific things (e.g., "how do I implement a recursive find function over immutable linked lists in JavaScript"). General bits of code only give you pieces of a solution, whereas specific bits of code often will give you a complete copy/pastable solution. If it's not 100% clear if something is permitted to be used or not, you can always ask me beforehand.

No discussion whatsoever is allowed during exams, except with the instructor. Any violations can result in a failing grade for the assignment/exam, or potentially failing the course for egregious cases. A report will also be made to the Dean of Academic Affairs. Students who repeatedly violate this policy across multiple courses may be suspended or even expelled.

LLM Policy

For assignments, LLM-based tools like ChatGPT are discouraged, though not outright disallowed. It can be easy to over-rely on such tools without realizing it, to the point

where the tool does all the work, with zero understanding of what it produces. If you do use such tools, you should be aware of the following trap I've seen a lot of students fall into over the past two years:

- The student gets a solution from an LLM
- The student tests the solution, and observes that all tests pass
- The student looks at the code a bit, and decides that it "feels" right. This feeling is usually based on the relative length, complexity, formatting, and variable naming of the code, and not on its actual behavior.
- The student bombs a high-scoring, closely-related exam question, wherein the student was asked to write similar code for a similar problem.
- The student realizes they did not actually understand what the LLM produced, and then needs additional help to try to catch up.

Keep in mind that LLMs fundamentally just generate some text that statistically is likely to follow whatever the prompt is based on the training set, with no understanding of what the text says. Any output from an LLM **must** therefore be verified to ensure it is correct. Since we are learning new topics, you will unlikely be able to verify by looking at the code alone. They may still be useful for pointing you in some direction if you are lost, but there is no guarantee that the direction is even relevant (and I've helped students out of a completely irrelevant hole more than once).

One course-specific caveat to mention: LLMs will confidently provide misinformation when it comes to conceptual knowledge about programming languages, as there is a lot of publicly-available incorrect information when it comes to programming language design. This is particularly true for conceptual information about types.

Communication

In general, any questions should be made through Canvas. You can also email me, though I'm usually much faster to respond to Canvas than my general email. Office hours or talking with me after class are usually the fastest and most detailed way to communicate with me.

Late Policy / Exam Scheduling

Late assignments will be accepted without penalty if prior arrangements have been made or there is some sort of legitimate emergency (at my discretion). If you must be absent from an exam, contact me ASAP to see if alternative accommodations can be made.

If an assignment is otherwise submitted late, it will be penalized according to the following scale:

If your assignment is late by $\leq$ this many days...	...it will be deducted by...
1	10%

If your assignment is late by <= this many days...	...it will be deducted by...
2	30%
3	60%
4+	100%

To be clear, assignments which are submitted four or more days beyond the deadline will not receive credit. The reason for such a harsh late policy is that we will generally discuss solutions in class shortly after the deadline, and this late policy discourages people from simply pulling a solution from an in-class discussion.

Class Feedback

I am open to any questions / comments / concerns / complaints you have about the class. If there is something relevant you want covered, I can push to make this happen. I operate off of your feedback, and no feedback tells me “everything is ok”.

Class Schedule (Subject to Change):

Week	Monday	Wednesday
1	8/24: Introduction, motivation	8/26: OOP introduction with Java, using inheritance to avoid code duplication
2	8/31: OOP introduction with Java, using inheritance to avoid code duplication	9/2: Using inheritance to avoid code duplication, class-based inheritance, subtyping, virtual dispatch
3	9/7: Labor Day (no instruction)	9/9: Using inheritance to avoid code duplication, class-based inheritance, subtyping, virtual dispatch
4	9/14: Subtyping polymorphism and ad-hoc polymorphism	9/16: Interfaces in Java, reasoning about classes/interfaces
5	9/16: Functional/immutable lists: representation and operations, assignment 1 discussion	9/18: Functional/immutable lists: representation and operations, assignment 1 discussion
6	9/21: Midterm Exam 1 Review	9/23: Midterm Exam 1
7	9/28: Midterm Exam 1 Retrospective	9/30: JavaScript introduction, types introduction, static vs. dynamic typing

Week	Monday	Wednesday
8	10/5: Type Concepts: Strong vs. weak typing, undefined behavior, untyped languages, type inference	10/7: Type Concepts: Strong vs. weak typing, undefined behavior, untyped languages, type inference
9	10/12: Higher-Order Functions in JavaScript	10/14: Higher-Order Functions in JavaScript
10	10/19: Objects in JavaScript	10/21: Objects in JavaScript, Prototype-based Inheritance in JavaScript
11	10/26: Objects in JavaScript, Prototype-based Inheritance in JavaScript	10/28: Assignment 2 Discussion
12	11/2: Midterm Exam 2 Review	11/4: Midterm Exam 2
13	11/9: Midterm Exam 2 Retrospective	11/11: Veterans Day (no instruction)
14	11/16: Compilation vs. interpretation, Just-in-time compilation	11/18: Compilation vs. interpretation, Just-in-time compilation
15	11/23: Stack vs. Heap Allocation, Manual Memory Management	11/25: Reference Counting, Garbage Collection, Ownership Final Exam Review
16	11/30:, Structs in Rust	12/2: Introduction to Rust; Ownership and Borrowing in Rust
17	12/7: Buffer	12/9: Final Exam Review
18	12/14: No class; Final Exam for only Section 16798 in JD 1618 from 5:30 - 7:30 PM	12/16: No class; Final Exam for only section 16262 in Noski Auditorium from 10:15 AM - 12:15 PM